DRIVE: Dynamic Resource-Aware Intelligent Task Offloading in Vehicular Edge Computing

Buqing Cao, Zhi Tao, Shanpeng Liu, Hongfan Ye, and Jinjun Chen, *Fellow, IEEE*

***Abstract*—With the swift evolution of intelligent transportation systems, Vehicular Edge Computing (VEC) has developed rapidly. In VEC environments, the lack of dynamic awareness of spatiotemporal-varying heterogeneous resources results in high task offloading latency in high mobility scenarios. Therefore, this paper proposes an intelligent task offloading method based on dynamic resource awareness in vehicular edge computing environments by using Twin Delayed Deep Deterministic Policy Gradient (TD3). The proposed method first establishes a cloud-edge-end multi-dimensional collaborative framework to support the perception and coordination of unevenly distributed multi-dimensional dynamic resources throughout task offloading. Secondly, a multi-objective model is established with the core objective of minimizing task execution latency, while simultaneously considering computational and bandwidth resources as well as link timeliness constraints. Finally, the TD3 algorithm, which effectively mitigates Q-value overestimation bias through mechanisms such as clipped double Q-learning, is employed to derive efficient and rational task offloading decisions. Experimental results on datasets from Beijing and Shanghai telecom base stations and taxi trajectory, show that the proposed method achieves lower weighted task latency than existing baseline approaches in the field.**

***Index Terms*—Vehicular edge computing, collaborative computing, task offloading, TD3.**

I. Introduction

With the deep evolution of intelligent transportation systems, vehicular networks have become the core infrastructure supporting diverse applications including autonomous driving, vehicle-road collaboration, and in-vehicle entertainment [1]. Consequently, the volume of traffic-related data has experienced explosive growth, imposing more stringent requirements on the real-time performance and reliability of vehicular edge offloading [2]. However, although traditional cloud computing possesses massive resources, its deployment location far from end-users leads to significant transmission latency and network jitter, rendering it unable to meet the requirements of latency-sensitive applications. Meanwhile, Intelligent Connected Vehicles (ICVs), subject to constraints in size, energy consumption, and cost, exhibit inherent bottlenecks in computing and storage capabilities, thereby making it difficult for them to independently undertake complex tasks [3]. Vehicular Edge Computing, as a key technology bridging end devices and the cloud, offers a feasible pathway to resolving the aforementioned contradictions by deploying computational capabilities to Road Side Units (RSUs) and surrounding vehicles [4]. In VEC, task offloading can utilize the vehicle to infrastructure (V2I) mode to offload to edge servers (ESs) or the vehicles to vehicles(V2V) mode to offload to other ICVs. Employing a V2I-V2V integrated offloading approach (i.e., a hybrid offloading mode) can maximize resource utilization and enhance system resilience. However, the core bottlenecks of this model are as follows: 1) ICVs' high mobility leads to frequent changes in network topology, resulting in time-varying connectivity and unreliable multi-hop V2V links, which are prone to causing data transmission interruptions. 2) the service coverage of V2I is constrained by the deployment density of RSUs, leading to an uneven distribution of heterogeneous resource pools.

Fig. 1 depicts a task offloading scenario in a VEC environment that utilizes heterogeneous resources through collaborative mechanisms: ICV1, functioning as the client intelligent connected vehicle traveling on an urban expressway, initiates an autonomous driving environmental perception task during its operation. ICV1 first evaluates its own computational resources and, upon determining an inability to meet the task requirements, immediately triggers the task offloading process:1) In terms of cross-layer collaboration, ES1 serves as the ES within the communication range of ICV1. After ICV1 accesses ES1 through Edge-End collaboration, ES1 will undertake the task and allocate corresponding computing and bandwidth resources to ICV1.2) In terms of intra-layer coordination, ES2 serves as an adjacent server to ES1. If ES1's own computational resources are insufficient to meet the latency requirements of ICV1's autonomous driving environmental perception task, it initiates a

Received 30 March 2026; revised 26 June 2026; accepted 29 June 2026. Date of publication 3 July 2026; date of current version 5 October 2026. This work was supported in part by the National Natural Science Foundation of China under Grant 62572186, in part by the National Foreign Expert Program of China under Grant S20250241, in part by the Science and Technology Innovation Program of Hunan Province under Grant 2026RC9036, in part by the Open Project Funding of the Key Laboratory of Intelligent Sensing System and Security (Ministry of Education), and in part by the Scientific Research Fund of the Hunan Provincial Department of Education under Grant 25C0590. Recommended for acceptance by S. Ucar. *(Corresponding authors: Zhi Tao; Hongfan Ye.)*

Buqing Cao is with the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan 411201, China, and also with the School of Computer Science and Artificial Intelligence, Hunan University of Technology, Zhuzhou 412007, China (e-mail: buqingcao@gmail.com).

Zhi Tao is with the School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan 411201, China (e-mail: zhitao147@gmail.com).

Shanpeng Liu is with the School of Computer Science and Artificial Intelligence, Hunan University of Technology, Zhuzhou 412007, China (e-mail: liushanpeng0@gmail.com).

Hongfan Ye is with the School of Computer and Artificial Intelligence, Huaihua University, Huaihua 418000, China (e-mail: yxwdyhf@gmail.com).

Jinjun Chen is with the Department of Computing Technologies, Swinburne University of Technology, Melbourne, VIC 3122, Australia (e-mail: jchen@swin.edu.au).

Digital Object Identifier 10.1109/TMC.2026.3710011

1536-1233 © 2026 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies.
Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

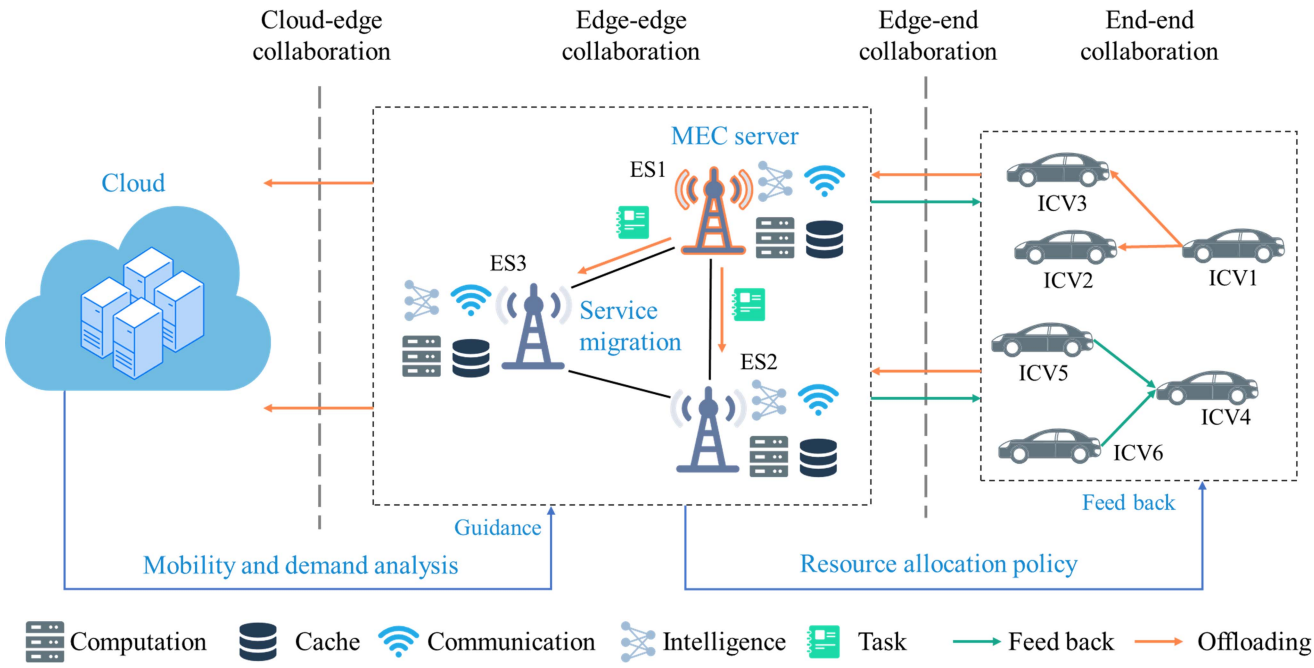


Fig. 1. Heterogeneous resource collaborative task offloading framework in VEC environment.

task migration request to the relatively idle ES2, migrating the autonomous driving environmental perception task to ES2 for continued execution. Simultaneously, ICV2 and ICV3 represent potential service ICVs within the communication range of ICV1. ICV1 range of ICV1. ICV1 will offload a subset of lightweight subtasks to both ICV2 and ICV3. Consequently, the distributed heterogeneous resource cluster formed by "ES1–ES2–ICV2–ICV3" achieves rational task scheduling and collaborative utilization of resources for VEC tasks through cross-layer collaboration and intra-layer collaboration.

In VEC environment, to achieve low-latency task processing and efficient resource utilization during task offloading, researchers have conducted extensive studies. In the field of V2I offloading, the work in [5] accomplishes preemptive scheduling of tasks for a single ES via the Lagrangian relaxation method. In contrast, literature [6] employs a game-theoretic model to optimize the competitive relationship between vehicles and ESs. However, such methods are often confined to single-node scenarios, presenting difficulties in addressing load balancing challenges across multiple servers. In the field of V2V offloading, the study in [7] explores the utilization of idle resources in parked vehicles. However, existing solutions primarily focus on single-hop vehicle resources and have yet to adequately exploit the collaborative computing potential of multi-hop vehicles. In the domain of intelligent optimization algorithms, Deep Reinforcement Learning (DRL) has emerged as a pivotal tool for dynamic VEC scenarios, leveraging its capabilities for sequential decision-making and environmental adaptation. Actor-Critic (AC), Deep Q-Network (DQN), and other DRL frameworks can handle continuous or discrete action spaces [8]. However, they have not fully considered the requirements for algorithm stability and dynamic resource adaptability in dynamic V2V-V2I scenarios, nor have they adequately incorporated the characteristics of time-varying links and resource heterogeneity, making it difficult to support precise offloading decisions. To address the above issues, this paper proposes a task offloading method for vehicular edge computing based on the Twin Delayed Deep Deterministic Policy Gradient [9], named DRIVE (**D**ynamic **R**esource-aware **I**ntelligent Task Offloading in **V**ehicular **E**dge Computing) which aims to achieve dynamic resource-aware offloading in the vehicular edge environment. This method first establishes a cloud-edge-end cross-layer collaborative architecture. Second, it constructs a multi-constraint model with the core objective of minimizing system-weighted task delay. Finally, it employs the TD3 algorithm adapted to dynamic VEC scenarios to solve for offloading decisions that accommodate high mobility and heterogeneous resource characteristics [10]. The key contributions of this paper include:

1) A multi-dimensional collaborative resource allocation framework is proposed, which jointly exploits cloud-edge-end cross-layer coordination and intra-layer cooperation to enable both dynamic awareness and integrated V2V-V2I offloading in the context of spatiotemporal-varying heterogeneous resources in high mobility VEC environments.
2) A TD3-based Markov decision process formulation that explicitly incorporates ICV mobility, time-varying computational/bandwidth resources, and link timeliness constraints to minimize weighted task execution latency through stable and adaptive offloading decisions.
3) Comprehensive validation on real-world datasets from Beijing and Shanghai, which demonstrates that DRIVE consistently achieves lower task latency than representative baselines.

The structure of this paper is organized as follows: Section I introduces the research background and core issues. Section II reviews related research work. Section III details the proposed DRIVE approach. Section IV presents the experimental results

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

and analysis. Section V discusses limitations and practical considerations. Finally, Section VI concludes the paper.

II. Related Work

As a core technology integrating intelligent transportation systems and edge computing [11], VEC's task offloading optimization have become a research hotspot in the academia. Existing research mainly focuses on four directions: V2I offloading, V2V offloading, mobility-aware offloading, and intelligent algorithm optimization.

V2I offloading enhances task processing efficiency by migrating in-vehicle tasks to ESs deployed on RSUs, leveraging the computational resources of edge nodes. Early research focused primarily on resource scheduling in single-ES scenarios. Early research primarily focused on resource scheduling in single-ES scenarios: Sorkhoh et al. [5] designed a Lagrangian relaxation-based task preemptive scheduling strategy that addresses the temporal characteristics of vehicle-edge server connections. This approach dynamically adjusts task priorities to reduce system latency while ensuring a high admission rate for high-priority tasks. Wang et al. [6] modeled the competition for edge resources among vehicles as a non-cooperative game, maximizing individual utility functions by adjusting the task offloading probability, although their assumption of unlimited ES resources did not account for practical load constraints. With the deployment of multi-edge servers becoming mainstream, load balancing and service migration have emerged as key research priorities in V2I offloading. Dai et al. [12] used an optimization theory-based multi-server collaborative offloading strategy, which achieves system utility maximization and load balancing by selecting target ESs and optimizing task-splitting ratios. Zhang et al. [13] employed a DQN to optimize edge server selection and transmission mode decisions, transforming traditional non-convex optimization problems into an end-to-end learning framework via deep neural networks, thereby enhancing adaptability in dynamic environments. However, the aforementioned studies do not incorporate quantitative analysis of link disruption risks caused by vehicle mobility, making them difficult to adapt to highly-dynamic VEC scenarios.

V2V offloading effectively mitigates the issue of insufficient ES deployment density by utilizing idle computing resources of surrounding vehicles to establish a distributed collaborative computing network. Based on differences in vehicle states, existing research can be classified into static V2V offloading and dynamic V2V offloading: 1) In static scenarios, Li et al. [7] utilized a contract theory-based resource scheduling scheme for parked vehicles, which enhances the computing capacity of the edge layer by designing an incentive mechanism to encourage parked vehicles to share idle resources. Although such schemes can leverage stable static resources, their coverage is limited and struggles to meet the real-time demands of mobile vehicles. 2)Dynamic V2V offloading focuses on resource sharing among moving vehicles, requiring solutions to the challenges of link connectivity and resource dynamics. Ye et al. [14] treated each vehicle as an independent agent, employing deep reinforcement learning to enable autonomous selection of spectrum and power for V2V links, thereby reducing transmission overhead through decentralized decision-making. Qi et al. [15] integrated knowledge-driven approaches with Deep Reinforcement Learning to achieve multi-objective offloading decisions among vehicles, ESs, and base stations. Furthermore, Dai et al. [35] proposed a learning-based V2V computation offloading algorithm that enhances the awareness of server vehicles' capabilities and trustfulness, this approach effectively addresses the challenge of incomplete offloading information and ensures reliable V2V task processing by learning from historical performance without requiring prior global knowledge. However, existing dynamic V2V offloading schemes are mostly limited to single-hop vehicle resources, fail to fully tap the collaborative computing potential of multi-hop vehicles, and the lack of quantitative evaluation of multi-hop link connectivity also leads to insufficient task transmission reliability.

The high mobility of vehicles leads to frequent changes in network topology and time-varying link connectivity within the VEC environment. Effectively integrating mobility factors into offloading decisions has become crucial for ensuring service continuity. Yuan et al. [16] jointly optimized service migration and vehicle mobility, employing a multi-agent deep reinforcement learning algorithm to combat the high-dimensional "curse of dimensionality," thereby reducing latency while minimizing migration costs and travel time. Zhan et al. [17] used the Proximal Policy Optimization algorithm to balance long-term delay and energy consumption for mobile vehicle queue tasks, and adapted to location changes by dynamically adjusting the unloading timing. Liu et al. [18] presented a vehicular edge computing scheme that deploys ESs on mobile vehicles, integrating path planning and resource scheduling to overcome the limitations of static coverage. Liang et al. [19] considered the time-varying characteristics of wireless channels and employed hierarchical reinforcement learning to dynamically allocate resources, thereby enhancing offloading reliability. However, the aforementioned solutions predominantly assume the predictability of vehicle trajectories or overlook the interactions among multiple moving vehicles, and their robustness in complex traffic scenarios remains to be enhanced.

With the increasing complexity of VEC scenarios, traditional mathematical optimization methods are being superseded by intelligent algorithms due to their limitations in addressing dynamic and high-dimensional challenges, among which DRL has emerged as a mainstream approach by virtue of its sequential decision-making capability and adaptability to dynamic environments. In a single-agent scenario, He et al. [20] integrated the optimization objectives of network, caching, and computational resources, employing the DQN algorithm to solve the multi-objective offloading problem and leveraging experience replay to enhance efficiency. Liu et. al developed an A3C-based joint optimization algorithm in the research on multi-agent and enhanced DRL algorithms [21], which leverages multi-threaded parallel training and global parameter sharing to circumvent computational resource wastage caused by experience replay, thereby accelerating decision-making speed in dynamic environments. Wang et al. [36] introduced a multi-agent reinforcement learning framework that integrates attention mechanisms and

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

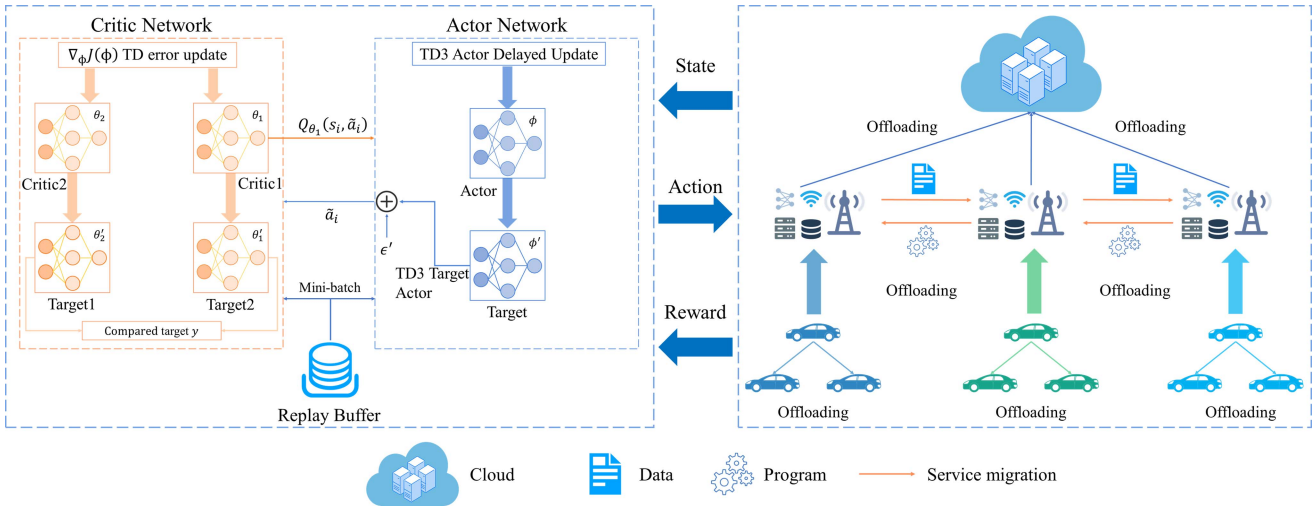


Fig. 2. The framework of DRIVE.

recurrent neural networks (RNN) to facilitate effective multi-agent coordination. This architecture enables agents to capture the temporal-spatial dependencies of vehicular states, optimizing joint offloading decisions through collaborative learning. Ye et al. [22] designed a willingness-aware DDPG offloading model (DELIGHT), which for the first time incorporates mobile terminals' offloading intent and ESs' service intent into the optimization objectives. Through the design of a multi-objective reward function, it achieves edge load balancing while reducing offloading costs. Wakgra et al. [37] developed a multi-agent TD3 approach to determine optimal offloading ratios in federated edge and vehicular-fog systems. Akbar et al. [34] proposed SaVE, a self-aware VEC framework that integrates vehicle self-awareness with VEXPSO, hierarchical federated learning, and DDPG to optimize resource allocation and minimize energy-latency costs in dynamic environments.

Through deeply studying on the aforementioned works and inspired by them, this paper utilizes the TD3 algorithm to propose a dynamic resource-aware task offloading method for vehicular edge computing (called DRIVE). Experimental results on the Beijing/Shanghai telecom base station datasets and Beijing/Shanghai taxi trajectory datasets demonstrate that DRIVE, while fully accounting for the high mobility of ICVs, the integrated V2V-V2I offloading characteristics, and dynamic fluctuations in heterogeneous resources, can significantly reduce the system-weighted task offloading latency, thereby delivering more reliable and efficient offloading decisions for dynamic VEC scenarios.

III. Models and Methods

The main mathematical symbols used in this section are shown in Table I.

A. Model Framework

As shown in Fig. 2, DRIVE employs a collaborative multi-resource allocation framework in VEC. This framework

TABLE I
Main Symbols Used In This Paper

Notation	Description
m	The number of vehicles
n	The number of ESs
s_i	The input size of task i
c_i	The number of CPU cycles required to process the task i
ξ_i	The delay constraint for task i execution
δ_i	The priority of task i
x_{ij}	The offloading decision
L_{ij}	The link connection time between node i and node j
r_{ij}^{V2V}	The data transmission ratio between ICV i and ICV j
$\bar{r}_{ij}^{V2V}(t)$	The average transmission rate between ICV i and ICV j
t_{ik}^{c}	The time that ICV k needs to process the offloaded task i
f_k	The computational capacity of ICV k
t_{ik}	The total time required for ICV k to complete the offloaded task i
t_i^{V2V}	The required minimum delay for processing the task i using V2V offloading
r_{ij}^{V2I}	The data transmission ratio between node i and ES j
$\bar{r}_{ij}^{V2I}(t)$	The average data transmission ratio of ICV i and ES j
t_{ij}^{u}	The delay that ICV i requires to transmit the task to the associated ES j
f_{ik}	The assigned computational capacity by ES k for task i
B_{ik}	The assigned bandwidth resource by ES k for task i
t_{ik}^{e}	The required delay for ES k to calculate the task i
t_{jk}^{mig}	The service migration time between ES i and ES k
t_{ik}^{V2I}	The total delay of processing task i by ES k
t_i^{cloud}	The time that cloud required to process the task i
u_i	The user utility

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

maximizes spatiotemporal-varying heterogeneous resource utilization through cross-layer collaboration and intra-layer collaboration, while leveraging the TD3 algorithm for offloading decisions to ensure the QoS for user ICVs. Specifically:

As shown on the right side of Fig. 2, the VEC environment in DRIVE comprises a central cloud, n ESs deployed throughout the city, and m ICVs in motion on the roads. The set of ESs and ICVs are illustrated as $N = \{1, \ldots, n\}$ and $M = \{1, \ldots, m\}$. All ESs are interconnected to the central cloud and other ESs via wired links, while ICVs access their associated ESs through V2I communication and achieve direct connectivity between ICVs via V2V communication. The ICV that initiates a computation task in the system is defined as client ICV. Each ES covers a subset of client ICVs, denoted as $M_j \in M (j \in N)$, within its communication range. The task of a client ICV i is characterized by a four-element set $\{s_i, c_i, \xi_i, \delta_i\}$, where s_i is the input task size, c_i stands for the number of CPU cycles required to process the task, ξ_i represents the latency constraint for task execution, and δ_i denotes the task priority. Let $x_{i0}, x_{ij}, x_{i(n+1)} \in \{0, 1\}$ (where $j \in M$ and $\sum_{j=0}^{n+1} x_{ij} = 1$) denoted offloading decision, specifying a task will be executed through V2V offloading, V2I offloading, or cloud computing. When $x_{i0} = 1$, it indicates V2V offloading, when $x_{ij} = 1$, it denotes that the task is offloaded via V2I to the j-th ES, and when $x_{i(n+1)} = 1$, it signifies cloud offloading.

As shown on the left side of Fig. 2, the Actor network of the TD3 model outputs a baseline offloading action based on current state provided by the VEC environment. The final action $\tilde{a}_i$ is obtained by superimposing an exploration noise ϵ onto this baseline action. The twin Critic networks evaluate the action value by computing the temporal difference error, while connecting with their respective Target networks to generate target actions for calculating the target Q-value, thereby mitigating Q-value overestimation. All transition samples (state, action, reward, next state) generated by the interaction between agents and the environment are stored in Replay Buffer, which facilitates batch sampling during training to improve data utilization efficiency. Meanwhile, to further ensure the reliability and adaptability of the decision-making, the Actor network is updated in a delayed manner, with its parameters optimized via the deterministic policy gradient $\nabla_\phi J(\phi)$ only after the Critic network has converged.

State Information Collection and Update Mechanism: To support dynamic resource-aware offloading, all ICVs and ESs periodically broadcast their local status information, including available computing resources, bandwidth resources, locations, and mobility states. The cloud scheduling center aggregates the received information and maintains a global system view. DRIVE adopts an event-driven decision mechanism. Specifically, an offloading decision is triggered only when an ICV generates a task that cannot be efficiently executed locally. After task execution, migration, or completion, the corresponding resource information is immediately updated to reflect the latest system state. This mechanism helps reduce the impact of stale information while avoiding unnecessary periodic offloading decision updates.

B. Mobility Model

The high mobility of ICV can easily cause link interruptions, leading to data transmission failures. Therefore, link connectivity is crucial for ensuring successful service offloading, and it can be represented by the connection duration of the link.

To quantitatively analyze the impact of ICV mobility on link connection time, this paper employs a free-flow traffic model to abstract the driving states of ICVs, similar to the approach in [24]. The ICV travels along the x-axis at a constant velocity. Assuming the initial position of ICV i is (p_{xi}, p_{yi}) and it travels in the positive direction at speed v_i. Based on this model, relevant definitions and derivations are as follows.

V2V Connectivity: Any two neighboring ICVs i and j are capable of establish communication with one another on the condition that the inter-vehicle distance $d_{ij}(t)$ falls within the communication range R_v. Based on the above connectivity rule, their link connection time can be expressed in the form given in reference [25]:

$$l_{ij} = \{\min t | d_{ij}(\tau) < R_v, 0 < \tau < t\} \tag{1}$$

To ensure the generality of subsequent derivations, this paper assumes that ICV j is ahead of ICV i. The link connection time under different conditions is derived as follows:

When ICV j and ICV i are traveling in the same direction:

If $v_j - v_i > 0$:

$$l_{ij} = \frac{R_v - (p_{xj} - p_{xi})}{v_j - v_i} \tag{2}$$

If $v_j - v_i < 0$:

$$l_{ij} = \frac{R_v + (p_{xj} - p_{xi})}{v_i - v_j} \tag{3}$$

When ICV j and ICV i are traveling in opposite directions:

$$l_{ij} = \frac{\sqrt{R_v^2 - (p_{yj} - p_{yi})^2} + (p_{xj} - p_{xi})}{v_i + v_j} \tag{4}$$

R_v is the communication range between ICVs, p_x and p_y representing the coordinate values in meters mapped within the Cartesian plane, v is the speed of ICV, and l_{ij} is the link connection time, i.e., connectivity, between ICV i and ICV j.

V2I connectivity: Assuming the coordinates of the ES k associated with the ICV i are (p_{xk}, p_{yk}). Then, the distance that ICV i must cover before moving out of the communication range of this ES is:

$$d_{ik} = \sqrt{R_r^2 - (p_{yi} - p_{yk})^2} - (p_{xi} - p_{xk}) \tag{5}$$

where R_r denotes the communication range of the ES. The duration that ICV i stays within the communication range of ES k can be derived based on (5) as:

$$l_{ik} = \frac{d_{ik}}{v_i} \tag{6}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

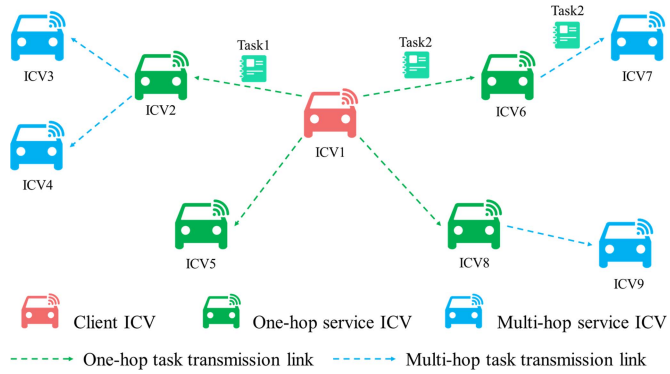


Fig. 3. V2V computation offloading mechanism.

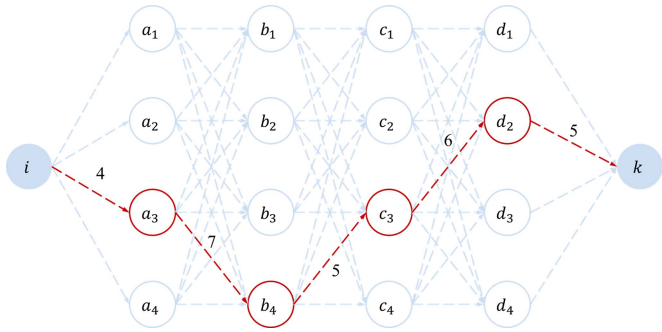


Fig. 4. The connectivity of multi-hop routing path.

C. V2V Offloading

As shown in Fig. 3, the red ICV represent client ICV, the green ICVs represent single-hop ICVs, and the blue ICVs represent multi-hop ICVs. For task processing of client ICV, in addition to local computation, pending tasks can be offloaded to surrounding available ICV nodes for execution. V2V offloading in this scenario can address the resource constraints of individual ICVs, allowing multiple ICVs to engage in distributed, collaborative task processing. The surrounding available ICVs are defined as other ICVs within the communication range of the client ICV, which possess idle computational resources and can establish connections via single-hop or multi-hop communication links. The discovery of available surrounding ICVs involves three key steps, which are service ICV discovery, routing path determination, and task allocation strategy. For example, in an urban road scenario, when a client ICV initiates a computation task offloading request: 1) When offloading Task1, if there are other eligible ICVs within the single-hop range, these ICVs become candidate service ICVs, in the routing phase, the single-hop link connectivity is evaluated to ensure end-to-end communication, in the task allocation phase, by integrating the link transmission characteristics and the computing capability of the candidate ICVs, it is decided to offload it to ICV2. 2) When offloading Task2, if there is no suitable ICV within the single-hop range, the service discovery phase forms a multi-hop candidate service ICV set through request forwarding, in the routing phase, the optimal path is selected using the path connectivity of each multi-hop path as the metric, ultimately determining ICV7 as the service ICV, in the task allocation phase, the task is transmitted hop-by-hop to ICV7. The aforementioned V2V offloading example describes a complete procedural framework of service discovery, route determination, and task allocation. The details of this framework are as follows:

1) Service ICV Discovery: Service requests are transmitted by the client ICV to other ICVs that lie within its single-hop and multi-hop communication ranges. These requests contain task attributes (such as task size) and service requirements (including computing resources and delay constraints). For ICVs that receive the service requests, they are required to perform two parallel operations: On one hand, when the predefined maximum hop count has not been reached, the service request continues to be forwarded to surrounding ICVs to expand the search scope for service ICVs. On the other hand, each ICV, based on its local resource status and the requirement parameters in the service request, judges whether it satisfies the criteria and whether it hosts necessary service entities for task processing. If these conditions are satisfied, it agrees to respond to the service request and sends the status information (e.g., computing capability) to client ICV, thereby becoming a candidate service ICV. An ICV can be identified as a valid service ICV if and only if the routing path connecting it to the client ICV remains connected until the computation result is successfully transmitted to the client ICV.

2) Routing Path Determination: Multiple potential routing paths may exist between the client ICV and its candidate service ICVs, with each path comprising a series of sequentially connected single-hop links. However, the high mobility of ICVs is prone to easily lead to frequent disruptions in vehicular network links [26]. Therefore, link connectivity is adopted as the primary criterion for routing selection. Specifically, routes exhibiting stronger and more persistent connections are preferred, since they increase the probability that computation results can be successfully delivered back to the client ICV, thus improving the overall robustness of task execution. As illustrated in Fig. 4, the client vehicle is denoted by i, while a candidate multi-hop service ICV is represented by k. There exist u multi-hop routing paths between, P is defined as the set of all these paths. For any path $p \in P$, consists of v single-hop links, specifically following the sequence, $o \rightarrow 1 \rightarrow 2 \rightarrow 3 \rightarrow \ldots \rightarrow v-1 \rightarrow k$. For routing path z, its connectivity equals the minimum value of the connection times across all its single-hop links, i.e.,:

$$p_z = \min\left\{l_{o1}, l_{12}, \ldots, l_{(v-1)k}\right\} \tag{7}$$

As shown in Fig. 4, in the path $i \rightarrow a_3 \rightarrow b_4 \rightarrow c_3 \rightarrow d_2 \rightarrow k$, the numbers labeled beside each link represent the link connection times. According to (7), the connectivity calculation result for this routing path is 4. Additionally, the optimal routing path p_{opt} refers to the path with the longest connection time among all candidate paths, specifically:

$$p_{opt} = \max_{z \in H}\{p_z\} \tag{8}$$

3) Task Allocation Strategy: Once the client ICV has collected all relevant information from the candidate service ICVs, it ought to determine the optimal computing offloading decision in accordance with the task service requirements and specific constraints. Task will send to a suitable service ICV for computation based on the offloading decision. After task execution,

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

the processed results are delivered back to the requesting ICV. The instantaneous data rate over the wireless link connecting adjacent ICVs i and j at time t can be expressed as:

$$r_{ij}^{V2V}(t) = B_{ij}^{V2V} log_2\left(1 + \frac{P_t g^{V2V}(t)}{\omega}\right) \tag{9}$$

where B_{ij}^{V2V} denotes the bandwidth between ICVs i and j, P_t represents the transmission power, $g^{V2V}(t)$ represents the channel gain that is able to further described as a function of distance between the two ICVs, ω signifies the noise power.

The mobility of ICVs causes the distance between any two ICVs to change over time, and this in turn leads to the transmission rate varying with time. Based on (9), the average transmission rate can be expressed in the form given in [27]:

$$\overline{r_{ij}^{V2V}(t)} = \frac{\int_0^{l_{ij}} r_{ij}^{V2V}(t)\, dt}{l_{ij}} \tag{10}$$

For the offloaded task size s_i, the time of single-hop transmission between ICVs i and j is obtained by:

$$t_{ij}^{one-hop} = \frac{s_i}{\overline{r_{ij}^{V2V}(t)}} \tag{11}$$

For a routing path made up of v single-hop links, specifically $o \to 1 \to 2 \to 3 \to \ldots \to v-1 \to k$, the data transmission time from client ICV i to service ICV k along this path is:

$$t_{ik}^{mul-hop} = t_{i1}^{one-hop} + \sum_{j=1}^{u-2} t_{j(j+1)}^{one-hop} + t_{(u-1)k}^{one-hop} \tag{12}$$

The time required for the service ICV k to execute task from the client ICV i is obtained as follows:

$$t_{ik}^{c} = \frac{c_i}{f_k} \tag{13}$$

where f_k denotes the computational capability of service ICV. Accordingly, the overall completion latency of the offloaded task can be formulated as:

$$t_{ik}^{V2V} = \begin{cases} t_{ik}^{one-hop} + t_{ik}^{c}, & k\ is\ one\ hop\ ICV \\ t_{ik}^{mul-hop} + t_{ik}^{c}, & otherwise \end{cases} \tag{14}$$

A candidate service ICV can only become a qualified service ICV for task processing if, throughout the entire process of successfully returning the computation results to the client ICV, the routing path between it and the client ICV maintains effective connectivity. All available candidate service ICVs constitute the set ϑ. Priority is given to selecting the ICV that yields the minimum task processing delay, i.e.,:

$$t_i^{V2V} = \min_{k \in \vartheta} \left\{ \frac{c_i}{f_i}, t_{ik}^{V2V} \right\} \tag{15}$$

where $\frac{c_i}{f_i}$ represents the local execution time of the offloaded task at client ICV i.

D. V2I Offloading

Task uploading, task execution, and result feedback are the three key phases of the task offloading process. Specifically, the client ICV is able to offload its tasks to the ES for execution, and this offloading process mainly consists of these three stages.

1) Task Uploading: Each client ICV with tasks to be offloaded, the task data to be processed must first be transmitted to its associated ES. The transmission delay is associated with factors like the bandwidth allocated by the ES and the distance between the ICV and the ES. Denote B_j as the total bandwidth resource allocated through orthogonal multiplexing among task ICVs within the coverage of ES j. Denote B_{ij} as the bandwidth allocated to ICV i. Therefore, the data transmission rate between ICV i and ES j is:

$$r_{ij}^{V2I}(t) = B_{ij}^{V2I} \log_2\left(1 + \frac{P_t g^{V2I(t)}}{w}\right) \tag{16}$$

where B_{ij}^{V2I} denotes the bandwidth between ICV i and the ES j, while $g^{V2I(t)}$ characterizes the corresponding channel gain, which varies with the separation distance between the two nodes. Moreover, for all client ICVs served by ES j, the aggregate allocated bandwidth is constrained by the maximum available bandwidth B_j, i.e.,

$$\sum_{i=1}^{M_j} B_{ij}^{V2I} \le B_j, \qquad j \in N \tag{17}$$

The distance between an ICV and the ES changes over time because of the mobility of ICVs, and this further results in a time-varying transmission rate [28]. Therefore, its average value is calculated as:

$$\overline{r_{ij}^{V2I}(t)} = \frac{\int_0^{l_{ij}} r_{ij}^{V2I}(t)\, dt}{l_{ij}} \tag{18}$$

The delay required for ICV i to transmit the task to its related ES j is:

$$t_{ij}^{u} = \frac{s_i}{\overline{r_{ij}^{V2I}(t)}} \tag{19}$$

2) Task Execution: The offloaded task of the client ICV i is uploaded to its nearby associated ES j. When receiving the offloaded task, ES j has two processing modes: local execution, or migration to other available ESs. Service migration typically occurs when current ES's computational capability cannot meet task requirements. Let k be the ES selected for task execution. Within the service range of the same ES, different client ICVs have to compete for the limited available channel resources. Each ES should reasonably allocate computational resources to tasks from different client ICVs. When providing services for multiple tasks, the chosen ES must efficiently allocate its limited computational resources among the various tasks. Let F_k denote the computational capability of the ES k, and f_{ik} represent the computational resources allocated to task i. Thus, the delay essential for the ES k to compute offloaded task i is:

$$t_{ik}^{e} = \frac{c_i}{f_{ik}} \tag{20}$$

Simultaneously, the sum of computational resources allocated to all users should not exceed the computational capability of ES k:

$$\sum_{i=1}^{M} f_{ik} \le F_k, k \in N \tag{21}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

If k and j represent different ES, an additional server migration time is incurred, specifically:

$$t_{jk}^{mig} = \begin{cases} 0, & j = k \\ h_{jk}\lambda, & j \neq k \end{cases} \tag{22}$$

where h_{jk} denotes the number of hops needed for task migration between ES i and ES j, and λ represents the time of single-hop migration. The total delay of V2I offloading to execute offloaded task i is defined as:

$$t_{ik}^{V2I} = t_{ij}^{u} + t_{jk}^{mig} + t_{ik}^{e} \tag{23}$$

3) Result Feedback: Once the computation is completed, the ES chosen to actually execute the task is responsible for sending results back to the client ICV. Because the data volume of computed outputs is substantially less than that of the initial input data, the transmission time required for returning the results is nearly negligible.

To ensure that the computation results can be successfully delivered to client ICV, t The connection duration of the client ICV with its associated ES shall not be surpassed by the total execution delay, specifically:

$$t_{ij}^{u} + t_{jk}^{mig} + t_{ik}^{e} < l_{ij} \tag{24}$$

E. Cloud-Edge-End Collaboration

Based on the actual location characteristics of the client ICV, cloud computing proves suitable for hosting delay-tolerant applications, whereas edge computing focuses on delivering delay-sensitive services. In cases where edge side resources cannot satisfy service requirements, cloud computing is leveraged as an auxiliary layer to compensate for the resource shortfall. The delay incurred by the cloud when processing the task from client ICV i primarily consists of transmission time between client ICV i and the correlative ES j, specifically t_{ij}^{u}, transmission time between ES j and the cloud, specifically as t_{jc}^{u}, the time required for cloud to execute the task, specifically as t_{ic}^{cloud}. Let f_i^{cloud} represent the computational capability allocated by the cloud, then:

$$t_{ic}^{cloud} = \frac{c_i}{f_i^{cloud}} \tag{25}$$

Therefore, the time required for cloud offloading of the task is:

$$t_{i}^{cloud} = t_{ij}^{u} + t_{jc}^{u} + t_{ic}^{cloud} \tag{26}$$

F. Problem Formulation

Subsequently, this paper converts the above analysis into an optimization problem which allow us systematically define the core objectives and constraints for resource allocation and task scheduling. In DRIVE, user utility can be characterized by their task processing delay:

$$u_i = x_{i0} t_i^{V2V} + \sum_{j=1\}}^{N} x_{ij} t_{ij}^{V2I} + x_{i(N+1)} t_i^{cloud} \tag{27}$$

Notably, the decision-making process in DRIVE follows an offline training and online inference paradigm. The computationally intensive training phase is executed on a centralized cloud server with substantial processing power, thereby eliminating real-time training latency. Furthermore, since the inference latency of the Actor network on ICV and RSU is typically much smaller than the millisecond-level task transmission and execution latency, this latency is omitted in the model. Additionally, the data size of execution results is typically much smaller than that of the input tasks, and the downlink bandwidth is generally sufficient to ensure near-instantaneous feedback, therefore, the return time for computation results to the ICVs is assumed to be negligible.

This paper's objective is to minimize the weighted sum of each individual user's utility. Let the task offloading strategy for client ICVs be denoted as $\boldsymbol{x} = x_{ij}, i \in M, j \in \{1, N, n+1\}$, the computational resource allocation strategy for ESs as $\boldsymbol{f} = \{f_{ij}, i \in M, j \in N\}$, and the bandwidth resource allocation strategy for ESs as $\boldsymbol{B} = \{B_{ij}, i \in M_j, j \in N\}$. The distributed resource collaboration awareness and task offloading optimization problem in VEC is formulated as the following constrained optimization problem:

$$\begin{aligned} \mathcal{P}1: \min_{\boldsymbol{x},\boldsymbol{f},\boldsymbol{B}} \ & \sum_{i=1}^{M} \delta_i u_i \\ \text{s.t.} \quad & C1: x_{ij} \in \{0,1\}, i \in M, j \in \{0, N, n+1\} \\ & C2 := 1, i \in \mathcal{N} \\ & C3: \sum_{i=0}^{M} f_{ij} \leq F_j, j \in N \\ & C4: \sum_{i=0}^{M_j} f_{ij} \leq B_j, \ j \in N \\ & C5: u_i \leq l_{ij}, \ i \in M_j, \ j \in N \end{aligned} \tag{28}$$

where δ_i is the task priority. An individual task may be processed via V2V, V2I, or cloud offloading, as stipulated by $C1$, a task can only be executed in one computing entity, namely an ICV, an ES, or the cloud, as ensured by $C2$. The constraint on computational resources and that on bandwidth resources are represented by $C3$ and $C4$, respectively. The task must be completed before the client ICV moves out of the communication coverage of its associated ES, as required by $C5$. Clearly, Problem $\mathcal{P}1$ is an NP-hard problem, Specifically, it involves a joint optimization of discrete offloading decisions x_{ij} and continuous resource allocation (f, B), which is a Mixed-Integer Non-Linear Programming (MINLP) problem. Since P1 can be reduced to the Generalized Assignment Problem (GAP), which is known to be NP-hard, solving it with traditional optimization methods presents significant challenges. DRL is considered an effective approach for addressing such complex optimization problems. Therefore, the subsequent section will employ the core DRL algorithm within the DRIVE framework to seek the optimal task and resource scheduling strategy [29].

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

G. Problem Solving

The TD3 algorithm, through mechanisms such as clipped double Q-learning, overcomes the inherent overestimation bias of Q-values and the resulting suboptimal decision-making problems found in previous DRL algorithms. Conversely, traditional optimization methods struggle to accurately capture the dynamic, stochastic, and time-varying characteristics of VEC networks, making them less effective for the joint optimization of computation offloading. Therefore, this paper selects the TD3 algorithm to make dynamic resource-aware offloading decisions in the VEC environment. The TD3 algorithm employed in DRIVE enhances stability and performance by effectively suppressing the overestimation of Q-values through the introduction of clipped double Q-learning, which use two independent Critic networks and compute the target Q-values. To implement this algorithm, the optimization problem needs to be reformulated as a Markov Decision Process (MDP). The tuple $\{\mathcal{A}, \mathcal{S}, \mathcal{R}, \mathcal{P}\}$ is what characterizes the MDP, where $\mathcal{A}$ represents the action space, $\mathcal{S}$ denotes the state space, $\mathcal{R}$ is used to represent the reward, and $\mathcal{P}$ signifies the transition probability. Specifically:

1) State Space: The state space must comprehensively reflect the resource status, task characteristics, and topological stability of the VEC network, specifically including the following dimensions: Edge Layer Resource Status: $F_{\mathrm{e}} = \{F_1, F_2, \ldots, F_n\}$, where F_j denotes the remaining computational capacity of the j-th ES, $F_{\mathrm{v}} = \{f_1, f_2, \ldots, f_m\}$: the remaining computational capacity of m ICVs, including idle resources from both single-hop and multi-hop service ICVs, $T_{task} = \{(s_i, c_i, \xi_i, \delta_i) | i \in M\}$: the set of tasks from all client ICVs, where s_i represents the task input size, c_i is the required CPU cycles, ξ_i is the latency constraint, and δ_i the task priority, $B = \{B_1, B_2, \ldots, B_n\}$: the remaining bandwidth resources of each ES, used for task transmission scheduling in V2I communication, $P_v = (p_{x1}, p_{y1}), \ldots, (p_{xn}, p_{yn})$: the current positions of all ICVs in the VEC network, $P_e = (p_{x1}, p_{y1}), \ldots, (p_{xm}, p_{ym})$: the positions of all ESs in the VEC network.

2) Action Space: In [38], the actor network outputs a continuous relaxed action vector with elements in $[0, 1]$, which is later mapped to feasible offloading decisions through a quantization process. Inspired by this, this paper also adopts a continuous action representation. Since the offloading decision $x_{i,j}$ is discrete, it is relaxed into a continuous representation in the action space and subsequently mapped to a feasible discrete decision. Specifically, the action generated by the agent is defined as a 3-dimensional continuous vector $a_i = [a_{i,0}, a_{i,1}, a_{i,2}] \in [-1, 1]^3$. The action is then mapped to the physical decision space through the mapping function $\mathcal{M} : a_i \rightarrow x_{i,j}, f_{i,alloc}, B_{i,alloc}$:

$$\mathcal{M} = \begin{cases} x_{i,j} = I\left(a_{i,0} \in \left[-1 + \frac{2J}{K}, -1 + \frac{2(J+1)}{K}\right)\right) \\ f_{i,alloc} = \alpha + \beta\left(a_{i,1} + 1\right) \\ B_{i,alloc} = \alpha + \beta\left(a_{i,2} + 1\right) \end{cases} \tag{29}$$

where $I(\cdot)$ is an indicator function that takes the value 1 when the condition inside the parentheses is true, and 0 otherwise. $x_{i,j}$ is the offloading decision mentioned in Section III-A, K is the number of unloading decisions $x_{i,j}$ with a value of n+2, and J is the discrete decision index. When $J = 0$, if $I(\cdot)$ is true, then V2V offloading are performed; when $1 < J \leq n$, if $I(\cdot)$ is true, then V2I offloading is performed and the J-th ES is unloaded; when $J = n + 1$, i f $I(\cdot)$ is true, then offloading to the cloud is performed, $f_{i,alloc}$ varies depending on the offloading decision and can be either the computational capability f_k of the service ICV mentioned in Section III-C or the computational resources $f_{i,k}$ allocated to task i mentioned in Section III-D, and $B_{i,alloc}$ differs based on the offloading decision, referring to either the bandwidth B_{ij}^{V2V} between ICVs i and j in Section III-C or the bandwidth B_{ij}^{V2I} between ICV i and the ES j in Section III-D, α and β are hyperparameters that control resource allocation.

3) Reward: Considering the defined action and state spaces, the reward function is designed to align with the objective of minimizing system latency. The reward r_i is formulated as:

$$\begin{aligned} r_i = & -ln\left(u_i\right) - \lambda \times \left(\max\left(0, c_i - f_{j,available}\right)\right. \\ & \left. + \max\left(0, B_{i,alloc} - B_{i,available}\right)\right) \end{aligned} \tag{30}$$

where $f_{j,available}$ represents the currently available computing resources for the offloading target j, $B_{i,available}$ represents the currently available bandwidth resources for the offloading target j,and λ is penalty factor of the penalty term, u_i denotes the weighted task latency of user i, obtained by multiplying the actual execution latency by the priority δ_i.

DRIVE utilizes the TD3 algorithm, employing a "twin Critic networks–single actor network–target networks" architecture to mathematically model the specific implementation steps in VEC.

Network initialization comprises two main components: the core network construction and the experience replay buffer setup. The core network construction involves building two independent Critic networks, denoted as $Q_{\theta_1}(s, a)$ and $Q_{\theta_2}(s, a)$, parameterized by θ_1 and θ_2, respectively. Both networks receive the current state s together with the corresponding action a and produce estimates of the state–action value, thereby alleviating overestimation bias. The corresponding target networks include $Q_{\theta_1'}(s, a), Q_{\theta_2'}(s, a), \pi_{\phi'}(s)$. The parameters are initialized with the parameters of the core networks, namely $\theta_1' = \theta_1$, $\theta_2' = \theta_2$, and $\phi' = \phi$, and are subsequently maintained for stability through soft updates. Experience replay buffer (Replay Buffer) is a buffer with an initial capacity of $\mathcal{U}_{max}$, used to store transition samples $(s_t, a_t, r_t, s_{t+1}, d_t)$ generated by the interaction between the agent and the environment, where d_t is termination state indicator, and $d_t = 1$ when the task is completed or a link interruption occurs.

To handle joint offloading and resource allocation within the standard TD3 framework, this paper formulates the decision-making process using a continuous action vector. As defined in Section III-G2), the Actor network outputs a three-dimensional action vector $a = [a_0, a_1, a_2]$. The Twin Critic networks $Q_{\theta_1}(s, a)$ and $Q_{\theta_2}(s, a)$ evaluate the corresponding state–action pairs, enabling gradient-based policy learning within the Actor–Critic framework. This continuous formulation avoids non-differentiable operations during training. During execution, the first component a_0 is deterministically mapped to the discrete offloading decision, while the remaining components a_1, a_2 correspond to continuous resource allocation variables.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: The Core Algorithm of DRIVE.

Initialization:
- Initialize core critic networks $Q_{\theta_1}(s,a)$, $Q_{\theta_2}(s,a)$ with parameters θ_1, θ_2, actor network $\pi_\phi(s)$ with parameter ϕ
- Initialize target critic networks $Q_{\theta'_1}(s,a)$, $Q_{\theta'_2}(s,a)$ with $\theta'_1=\theta_1$, $\theta'_2=\theta_2$, target actor network $\pi_{\phi'}(s)$ with $\phi'=\phi$
- Initialize replay buffer B with capacity $\mathcal{U}_{max}$
- Set hyperparameters: exploration noise std, smoothing noise std, clip range, discount factor γ, soft update coefficient τ, critic learning rate, actor learning rate
- Set thresholds: initial buffer size $\mathcal{U}_{init}$, update interval t_{update}, delay steps d_{delay}, batch size N_{batch}

Interaction and Experience Collection:
$t=0$
while training not terminated do
  Observe VEC state s_t
  Generate base action $a_t^{base}=\pi_\phi(s_t)$
  Sample exploration noise $\epsilon\sim\mathcal{N}(0,\sigma_{exp})$, compute $a_t=clip(a_t^{base}+\epsilon,0,a_{max})$
  Execute a_t, observe reward r_t, next state s_{t+1}, terminal flag d_t
  Store transition $(s_t,a_t,r_t,s_{t+1},d_t)$ in B
  $t=t+1$

Network Training and Update:
if $|B|\geq\mathcal{U}_{init}$ and $t\%t_{update}==0$ then
  Sample batch $\{(s_i,a_i,r_i,s'_i,d_i)\}_{i=1}^{N_{batch}}$ from B
  Target Action and Q-Value Calculation:
  for each sample in batch do
    Generate target action $a'_i=\pi_{\phi'}(s'_i)$
  Sample smoothing noise ϵ', compute $\widetilde{a'_i}=a'_i+\epsilon'$
    if $d_i==0$ then
      $y_i=r_i+\gamma\cdot\min(Q_{\theta'_1}(s'_i,\widetilde{a'_i}),Q_{\theta'_2}(s'_i,\widetilde{a'_i}))$
    else
      $y_i=r_i$
    end if
  end for
  Critic Network Update:
  Update θ_1 via minimizing $\frac{1}{N_{batch}}\sum_{i=1}^{N_{batch}}(y_i-Q_{\theta_1}(s_i,a_i))^2$
  Update θ_2 via minimizing $\frac{1}{N_{batch}}\sum_{i=1}^{N_{batch}}(y_i-Q_{\theta_2}(s_i,a_i))^2$
  Delayed Actor Network Update:
  if $t\%d_{delay}==0$ then
    Compute policy gradient $\nabla_\phi J(\phi)$
    Update ϕ via gradient ascent
    Target Network Soft Update:
    $\theta'_1\leftarrow\tau\cdot\theta_1+(1-\tau)\cdot\theta'_1$
    $\theta'_2\leftarrow\tau\cdot\theta_2+(1-\tau)\cdot\theta'_2$
    $\phi'\leftarrow\tau\cdot\phi+(1-\tau)\cdot\phi'$
  end if
end if
end while

TABLE II
EXPERIMENTAL SIMULATION PARAMETERS

Description	Value
ICV transmission range	500m
ES transmission range	1000m
Transmission power	23dbm
Noise power	-30dbm
Computational capacity of ICV	[200,800] Mcycles/s
Computational capacity of ES	[1,5] Gcycles/s
The cycles required for a task	[20,500] Mcycles

Each time step t, TD3 algorithm interacts with the VEC environment and collects experience. State observation refers to the agent acquiring the current VEC network state s_t, which includes resource status and task queues, among other factors. The action generation involves the Actor network $\pi_\phi(s_t)$ outputting a base action a_t^{base}, which is then augmented with an exploration noise $\epsilon\sim\mathcal{N}(0,\sigma_{exp})$ to ensure exploratory behavior, where σ_{exp} denotes the deviation of exploration noise. Action is clipped to ensure adherence to resource constraints. The environmental feedback, upon executing action a_t, involves observing the task execution delay u_t, the reward r_t, the next state s_{t+1}, and the termination indicator d_t. Experience storage entails saving the transition sample $(s_t,a_t,r_t,s_{t+1},d_t)$ into the experience replay buffer.

Network training and parameter updates are initiated once the sample count in the replay buffer reaches the threshold $\mathcal{U}_{init}$, and are performed every t_{update} steps. The process begins with sampling and target value calculation: a batch of transition samples $\{(s_i,a_i,r_i,s'_i,d_i)\}_{i=1}^{N_{batch}}$ with batch size N_{batch} is sampled from the experience replay buffer randomly, and target Q-value y_i is computed. Target action generation involves producing an action $a'_i=\pi_{\phi'}(s'_i)$ from the target state s'_i via the target Actor network, to which target policy smoothing noise $\epsilon'\sim clip(\mathcal{N}(0,\sigma_{smooth}),-c,c)$ is added, where σ_{smooth} is the standard deviation of the target policy smoothing noise and c denotes the clipping range. The smoothed target action $\tilde{a}_i=a'_i+\epsilon'$ is ultimately obtained to mitigate value fluctuations caused by function approximation errors. Target Q-value calculation utilizes the minimum output from two target Critic networks to suppress value overestimation, given by:

$$y_i=\begin{cases}r_i+\gamma\cdot\min_{k=1,2}Q^u_{\theta_k}(s'_i,\tilde{a}'_i) & \text{if } d_i=0\\ r_i & \text{if } d_i=1\end{cases}\tag{31}$$

where $\gamma\in(0,1]$ denotes the discount coefficient that weights long-term returns against instantaneous rewards, and $Q^u_{\theta_k}(s'_i,\tilde{a}'_i)$ represents the value predicted by the target Critic for the subsequent state–action pair.

As given by the formula, the Critic parameters are optimized by reducing the squared discrepancy between the target value y_i and the outputs of the current Critic estimators, resulting in updated parameters θ_1 and θ_2:

$$\theta_k\leftarrow\arg\min_{\theta_k}\frac{1}{N_{batch}}\sum_{i=1}^{N_{batch}}(y_i-Q_{\theta_k}(s_i,a_i))^2\ (k=1,2)\tag{32}$$

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

where $Q_{\theta_k}(s_i, a_i)$ represents the Q-value estimate of the k-th Critic network for state s_i and action a_i in i-th sample.

Then, the delayed Actor network update is performed every d_{delay} steps to avoid policy update bias when the value estimation has not yet converged. The update direction follows the deterministic policy gradient aimed at maximizing the expected output of the Critic network $Q_{\theta_1}(s, a)$, given by the formula:

$$\nabla_\phi J(\phi) = \frac{1}{N_{batch}} \sum_{i=1}^{N_{batch}} \nabla_a Q_{\theta_1}(s_i, a)|_{a=\pi_\phi(s_i)} \cdot \nabla_\phi \pi_\phi(s_i) \tag{33}$$

The term $\nabla_\phi J(\phi)$ corresponds to the sensitivity of the objective function to variations in the policy network parameters ϕ. The objective function $J(\phi)$ corresponds to the expected state-action value, as evaluated by the Critic network, under the policy π_ϕ. This gradient is computed to update the policy network parameters via gradient ascent, thereby improving the policy. The term $\nabla_a Q_{\theta_1}(s_i, a)|_{a=\pi_\phi(s_i)}$ represents the sensitivity of the Critic function Q_{θ_1} to variations in the action a variable, evaluated at the action output by the policy network $\pi_\phi(s_i)$ given the state s_i. This term captures how variations in the selected action affect the estimated value. Meanwhile, $\nabla_\phi \pi_\phi(s_i)$ reflects how the policy output responds to changes in its parameterization at state s_i. Based on these gradients, the policy parameters are optimized through a gradient ascent procedure, with Adam adopted to promote stable and smooth learning. This process leverages the adaptive gradient adjustment capability of the Adam optimizer, enabling the Actor network to adjust its parameters smoothly based on the value assessment from the Critic network, thereby avoiding drastic fluctuations during policy updates. Consequently, more robust policy learning and iteration are achieved in the dynamically changing VEC environment. To preserve learning stability, a soft update mechanism is applied to the target models. After each policy update, the corresponding target parameters are gradually adjusted according to the following rule:

$$\begin{aligned} \theta'_k &\leftarrow \tau \cdot \theta_k + (1-\tau) \cdot \theta'_k \, (k = 1, 2), \\ \phi' &\leftarrow \tau \cdot \phi + (1-\tau) \cdot \phi' \end{aligned} \tag{34}$$

where τ is soft update coefficient, used to gradually track changes in the core network parameters and avoid excessive fluctuations in the target values. Algorithm 1 provides a detailed description of DRIVE. The description is as follows: 1) During the initialization phase, initialization is carried out on the parameters of the core Critic networks $Q_{\theta_1}(s, a)$ and $Q_{\theta_2}(s, a)$, Actor network $\pi_\phi(s)$ as well as their corresponding target Critic networks $Q_{\theta'_1}(s, a), Q_{\theta'_2}(s, a)$, and the target Actor network $\pi_{\phi'}(s)$. At the same time, the experience replay buffer B is initialized, and various hyperparameters and thresholds are configured. 2) Interaction and Experience Collection (Lines 2-8): For each time step, the agent acquires the current state s_t of VEC environment, a base action a_t^{base} is generated by the Actor network, superimposes exploration noise to obtain the final action a_t and executes it. Subsequently, it observes the reward r_t, the next state s_{t+1}, and the termination indicator d_t, then stores the transition sample into the replay buffer B. 3) As soon as the sample count in the replay buffer reaches the threshold $\mathcal{U}_{init}$ and the update interval condition t_{update} is met (Line 9), batch samples are sampled from the buffer. For each sample, a target action is generated via the target Actor network with added smoothing noise (Lines 10–20). Then, depending on whether it is a terminal state, the target Q-value y_i is calculated using the minimum output of the two target Critic networks. 4) In the process of minimizing the mean squared error between the target Q-value and the outputs of the current Critic networks, the parameters of the core Critic network $Q_{\theta_1}(s, a)$ and $Q_{\theta_2}(s, a)$ are updated (Lines 21–23). If the delayed update step condition d_{delay} is met, the gradient of the Actor network is computed based on the deterministic policy gradient, and the Actor network parameters ϕ are updated via gradient ascent (Lines 24–27). 5) Following each update to the Actor network, the parameters of the target Critic network $Q_{\theta'_1}(s, a)$, $Q_{\theta'_2}(s, a)$, and the target Actor network $\pi_{\phi'}(s)$ are softly updated (Lines 28–31) to maintain the stability of the target networks.

IV. Experiments

A. Experimental Setup and Baseline Methods

This paper employs the Beijing/Shanghai telecom base station datasets and the Beijing/Shanghai taxi trajectory datasets to comprehensively validate the model. The specific settings of the datasets are as follows:

- *Edge Servers:* In the experiments, the base data for the ESs are sourced from the Beijing/Shanghai telecom base station datasets. These datasets cover the geographical distribution and service time information of the telecom mobile communication base stations in Beijing and Shanghai. The remaining computational resources of each ES are randomly generated based on a normal distribution within a specified range according to the experimental requirements.
- *ICV:* The positional data of ICVs utilize the Beijing /Shanghai taxi trajectory datasets, which contain real-time GPS trajectory information for 10357 taxis in Beijing and 4316 taxis in Shanghai, respectively. During the experiment, two random points in each taxi trajectory are selected as the locations where ICVs perform task offloading, thus generating the ICV positions and their corresponding tasks in Shanghai and Beijing. According to the experimental requirements, the computational resources required by the ICVs' tasks and the remaining computational resources of the ICVs are randomly generated within a specified range based on the normal distribution.

The experiments in this paper were conducted on a hardware setup with Intel Core i9-14900K processor, an NVIDIA RTX 4080 Super GPU, and 64 GB of RAM. Software environment utilized Windows 11, Python 3.11, and PyTorch 2.1. The learning rates were set to 0.0001 for the Actor network and 0.0003 for the Critic network, with γ at 0.99. Key simulation parameters are summarized in Table II.

The baseline methods in this paper are as follows:

- *Random:* The client ICV randomly selects an ES or a service ICV to offload its task request.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

- *Genetic Algorithm (GA) [30]:* This algorithm mimics the biological processes of mutation, inheritance, and selection that underpin natural selection and population reproduction, iteratively searching the solution space for the service offloading problem in the VEC environment to approximate the optimal offloading decision scheme.
- *Immune Algorithm (IA) [31]:* The Immune Algorithm draws inspiration from the antigen recognition, antibody generation, and immune regulation mechanisms of the biological immune system, while also incorporating concepts from genetic evolution. Its process for solving the service offloading problem in the VEC environment is as follows: first, it identifies the problem objective and generates initial candidate solutions. Second, it evaluates the quality of the antibodies using a fitness function. Finally, it employs various immune operations to optimize the candidate solutions, progressively searching for the optimal offloading strategy.
- *DQN [32]:* The DQN algorithm integrates traditional Q-learning with deep neural networks to solve sequential decision-making problems. In the service offloading scenario within the VEC environment, its core logic is as follows: leveraging the current state of the environment and the agent, the function $Q(s, a; w)$ generated by the deep Q-network is used to approximate the action value $Q(s, a)$, an $\epsilon - greedy$ strategy is employed to balance exploration and exploitation, output actions, and make decisions, the network parameters are updated according to the reward signals fed back by the environment, and the decision-making process is iteratively optimized until it converges to a stable policy.
- *DDPG [33]:* The DDPG algorithm combines the Deterministic Policy Gradient with deep neural networks to address sequential decision-making problems in continuous action spaces. In the service offloading scenario within the VEC environment, its core logic is as follows: it employs an Actor-Critic architecture where the Actor network directly maps the current environment state to a deterministic action, while a single Critic network approximates the action-value function $Q(s, a)$ to evaluate the policy's performance. To balance exploration and exploitation, action space noise is integrated into the decision-making process; the network parameters are then updated using Experience Replay and Target Networks based on the reward signals, and the policy is iteratively refined until it converges to an optimal, stable strategy for complex resource allocation.

In VEC scenarios, task processing delay is not only a hard constraint metric for real-time applications (e.g., autonomous driving), but also a comprehensive reflection of multi–dimensional optimization effects, including V2V multi–hop link connectivity, V2I service migration efficiency, and cloud–edge–end collaborative decision–making and resource allocation. Moreover, it serves as a prerequisite for optimizing other metrics such as resource utilization and energy consumption. Therefore, this paper adopts task processing delay as the sole evaluation indicator.

To quantify the inference overhead of the deployed Actor network, we measured the latency over 10 000 calls on the experimental platform, yielding an average per-decision latency of 0.12 ms. This is more than three orders of magnitude smaller than the average task processing delays observed in our experiments, confirming that omitting inference latency from the model is reasonable.

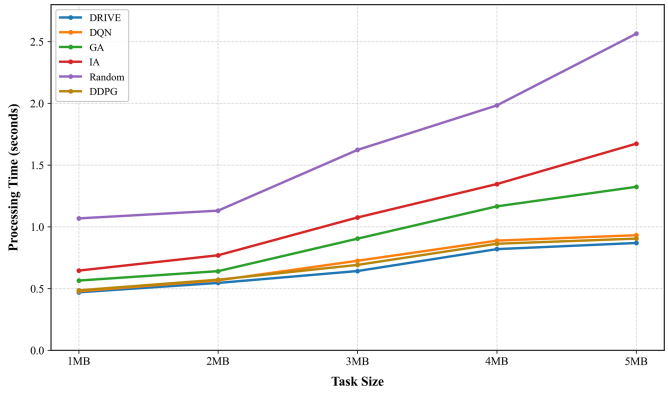


Fig. 5. Average task processing time of different offloading methods under the Beijing dataset.

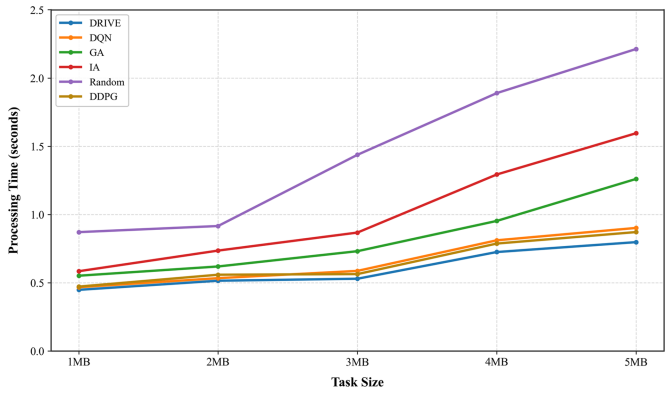


Fig. 6. Average task processing time of different offloading methods under the Shanghai dataset.

This paper conducts comparative experiments with task sizes ranging from [1], [5] MB in increments of 1 MB, evaluating the task processing performance of DRIVE, DQN, GA, IA, and Random in both the Beijing and Shanghai scenarios.

B. Experimental Results and Analysis

The outcomes of the experiments are illustrated in Figs. 5 and 6, where the x-axis corresponds to task size and the y-axis indicates the mean processing latency per task (unit: s). As shown in the figures, with the increase in task scale, the processing time of all comparative algorithms exhibits an increasing trend, but with significant differences in performance.

1) Analysis of Random and Heuristic Algorithms: The processing time of the Random algorithm consistently remains at the highest level. Especially when the task size reaches 3 MB, it increases by 43.5% (56.97%) under the Beijing (Shanghai) dataset compared with the scenario where the task size is 2 MB. This is because the random strategy lacks collaborative scheduling and planning for task characteristics and resource status, and

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

cannot dynamically match the optimal processing entity according to task complexity. As a result, its efficiency disadvantage becomes particularly prominent when the task data volume is large. In contrast, heuristic algorithms such as IA and GA exhibit a more gradual growth trend in processing time, indicating that these algorithms achieve better task allocation performance than the random strategy through predefined heuristic rules. Taking the 5 MB task as an example, under the Beijing (Shanghai) dataset, the average processing time of IA is reduced by 34.73% (27.89%) compared with the Random algorithm, and that of GA is reduced by 48.37% (42.97%) compared with the Random algorithm. However, their processing time is still significantly higher than that of DRL-based algorithms. Taking the 5 MB task size as an example, under the Beijing (Shanghai) dataset, the average processing time of IA and GA is 37.82% (36.86%) higher than that of DQN and 42.01% (44.15%) higher than that of DRIVE. This gap highlights the fundamental limitations of heuristic algorithms in meeting the requirements of distributed resource collaborative management and dynamic scheduling in VEC scenarios: these methods usually rely on predefined rules or static optimization paradigms, and lack real-time perception and response capabilities to key challenges such as time-varying connectivity of V2V multi-hop links, uneven V2I service coverage, and dynamic fluctuations of spatiotemporal-varying heterogeneous resource pools. Consequently, they struggle to achieve efficient resource integration and dynamic adaptation of task scheduling.

2) DRL Algorithm Analysis: In DRL algorithms, DRIVE, DQN and DDPG demonstrate superior performance over heuristic methods. However, DRIVE maintains the lowest processing time across all task scales and exhibits the most gradual growth in processing time. In the 1 MB task scenario, all DRL algorithms achieve excellent performance with negligible differences among them. As the task scale increases, in the 5 MB task scenario, based on the Beijing (Shanghai) dataset, the processing time of the DRIVE algorithm is reduced by 7.23% (13.05%) compared with the DQN algorithm, and by 4.08% (9.25%) compared to DDPG. This result can be explained from an algorithmic perspective: the Actor-Critic network collaborative optimization paradigm of DRIVE is more suitable for the decision-making requirements of VEC environment in high-dimensional state spaces. In contrast, as a DRL method based on Q-learning, DQN is susceptible to estimation bias and the exploration-exploitation trade-off in complex and dynamic VEC environments, resulting in limited decision-making accuracy and convergence efficiency. Due to DDPG's inherent deterministic nature and the lack of a twin-critic mechanism, is prone to severe Q-value overestimation and its sensitivity to hyperparameter settings in highly dynamic V2I topologies, its performance is inferior to that of DRIVE with TD3. Notably, DQN slightly outperforms DDPG when the task size is less than 3 MB. This may be because the discrete action mapping of DQN reduces the complexity of the search space, enabling it to converge faster than the continuous policy gradient method of DDPG when the system state is relatively stable and the load is light. In summary, by constructing a multi-dimensional collaborative resource management framework and adopting a twin Critic network and delayed policy update mechanism, DRIVE significantly improves the optimization capability of heterogeneous resource integration.

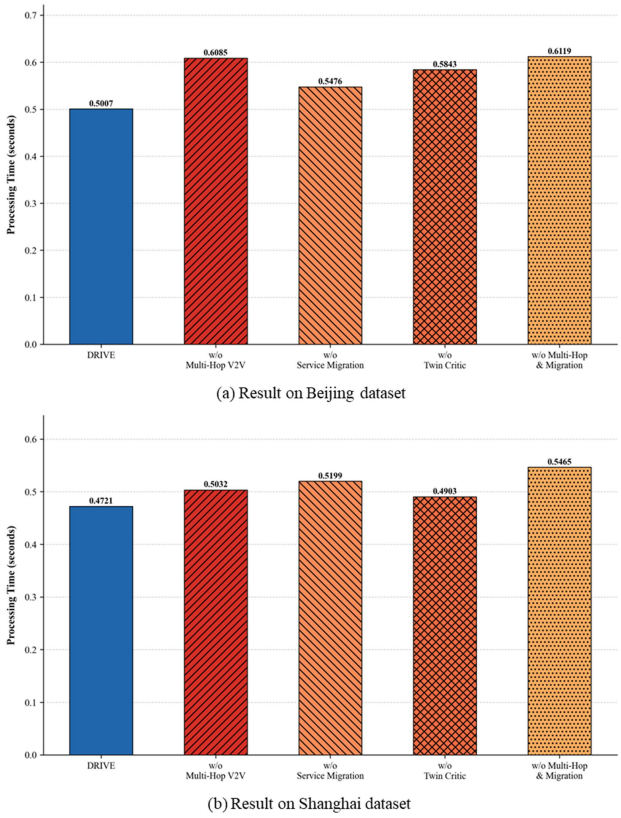


Fig. 7. Ablation experiment result.

3) Ablation Experiment Analysis: To further verify the contribution of each key component within the DRIVE framework, this paper conducts a comprehensive ablation analysis. In the ablation experiment, the task size follows a normal distribution with a mean of 3 MB and a variance of 0.8.

As shown in Fig. 7, the impact of multi-hop V2V and service migration is most significant. When the multi-hop V2V communication capability is disabled, the processing time increases to 0.6085 s(0.5032 s) in the Beijing (Shanghai) dataset, representing a performance degradation of 21.52% (6.58%). Similarly, in Beijing (Shanghai) dataset, removing the service migration leads to a rise in processing time to 0.5476 s (0.5199 s). resulting in performance degradation of 9.36% (10.12%). When both multi-hop V2V and service migration are disabled, the processing time increases to 0.6199(0.5465), corresponding to a performance degradation of 23.8% (15.7%). These results underscore that multi-hop relaying expands the manageable resource pool beyond a single-hop range, while service migration allows task continuity amidst high vehicle mobility. Without these components, the system is forced to rely on local or single-hop resources, which are easily congested, thereby increasing delays.

The effectiveness of the Twin Critic structure is also evident. Compared to the full DRIVE, removing the Twin Critic increased the processing time for the Beijing (Shanghai) dataset to

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

0.5843 (0.4903), an increase of 20.8% (4.8%). This performance gap is more pronounced in the Beijing dataset primarily because it features a higher density of base stations and vehicles, creating a significantly more complex environment. This complexity leads to higher-dimensional state spaces and continuous action spaces, where the Twin Critic architecture plays a critical role in mitigating overestimation bias, which is particularly evident in complex VEC environments.

C. Convergence Analysis

In service offloading tasks within the VEC environment, the selection of hyperparameters has a vital influence on the granularity and the convergence within the algorithm's learning process. This study conducts an in-depth analysis of the influence of the discount factor γ and learning rate lr on the loss function. In multiple experiments, the DRIVE algorithm converges after approximately 150 episodes. Based on the experimental results, this study adopts a configuration of low learning rates and a discount factor close to 1 ($Actor_lr = 0.0001$, $Crital_lr = 0.0003$, $\gamma = 0.99$). This configuration allows the DRIVE algorithm to capture the local details of gradient updates more thoroughly during training, all while guaranteeing the model's convergence and stability. Although this prolongs the training cycle to a certain extent, the resulting performance gain is highly significant from the perspective of long-term convergence results.

Fig. 8 depict the training loss curves of the Actor and Critic components. As observed, both losses exhibit pronounced oscillations during the initial 50 training episodes, followed by a gradual convergence after approximately 100 episodes, eventually stabilizing near their optimal levels after 150 episodes. Among them, the Critic network needs to continuously approximate the true Q-value through temporal difference errors, and its loss exhibits short-term fluctuations at around 150 episodes and 250 episodes. The speculated reasons include noise introduced by target policy smooth regularization, phased biases in value function estimation, and the combined influence of the periodic delayed update strategy of the target Actor network. However, benefiting from the low learning rate setting, as the training process progresses, although the Critic network still fluctuates around 250 episodes, it can reduce these biases through fine-grained gradient updates, and finally achieves stable convergence of the overall network.

D. Statistical Performance Analysis

To evaluate the stability of the proposed algorithm and mitigate the stochastic effects induced by random seeds, we conducted rigorous statistical experiments on the Shanghai dataset. The experimental configuration was defined as follows: the task data size followed a normal distribution with a mean of 3 MB and a variance of 0.8. For the DRIVE, DDPG, and DQN algorithms, we performed 10 independent trials using 10 distinct random seeds (0–9) and recorded the task latency metrics. Table III presents the statistical results in the form of mean $\pm$ standard deviation. The experimental data indicate that DRIVE not only achieves optimal average performance but also significantly outperforms the baseline algorithms in terms of stability.

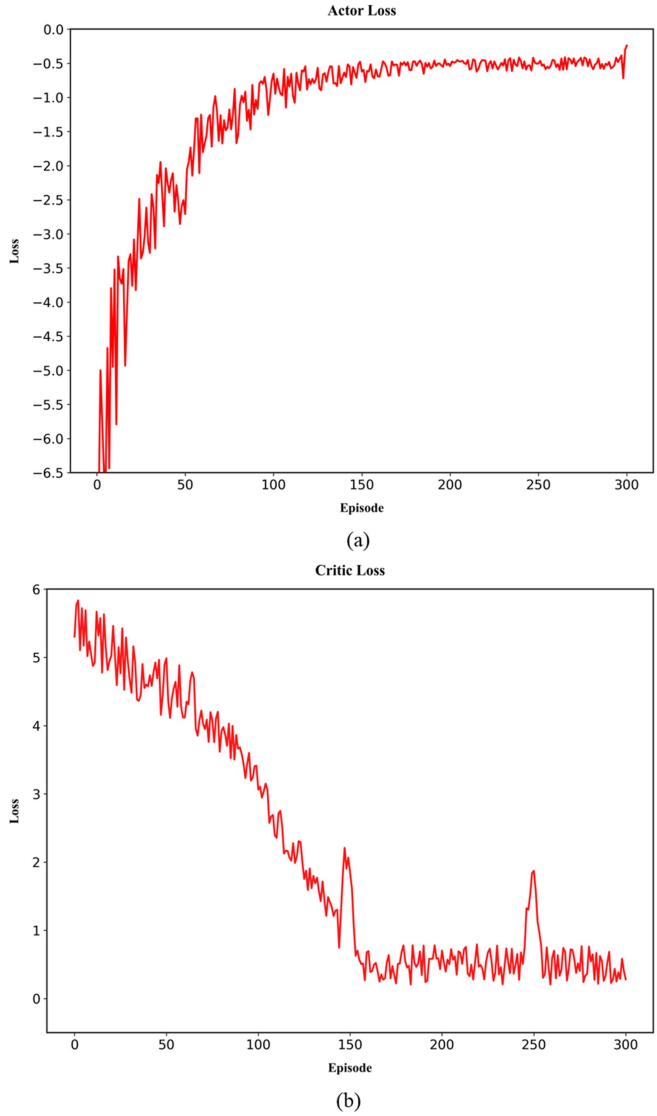


Fig. 8. Actor loss and critic loss.

TABLE III
STATISTICAL PERFORMANCE COMPARISON

Model	DRIVE	DDPG	DQN
Mean±std	0.504±0.037	0.529±0.052	0.574±0.071

The average latency of DRIVE is 0.504s, representing a reduction of 4.7% compared to DDPG (0.529s) and 12.2% compared to DQN (0.574s). Furthermore, the standard deviation of DRIVE is a mere 0.037, which is 28.8% lower than that of DDPG (0.052) and 47.9% lower than that of DQN (0.071). These results demonstrate that DRIVE maintains the lowest average latency while exhibiting superior robustness against disturbances and higher convergence consistency. In contrast, DQN exhibits the highest volatility due to the constraints of a discrete action space and high variance in value function estimation. While DDPG demonstrates proficient continuous control capabilities, its single-critic architecture remains susceptible to overestimation bias, resulting in stability that is marginally inferior to DRIVE.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

V. Discussion

The proposed framework adopts a simplified mobility abstraction, modeling vehicle movement as approximately constant-velocity within each decision interval. While this assumption enables tractable estimation of V2V link duration and V2I residency time, it does not explicitly capture several traffic dynamics commonly observed in urban environments, such as acceleration/deceleration, lane changes, temporary link blockages, and stop-and-go congestion. These factors may introduce additional uncertainty in link stability and resource availability.

The real-world taxi trajectory datasets collected from Beijing and Shanghai partially reflect such dynamics, as the traces inherently contain speed fluctuations and congestion patterns arising from urban traffic conditions. Furthermore, the mechanism of DRIVE allows the framework to adapt to mobility-induced variations. When mobility dynamics lead to localized resource shortages, tasks can be migrated to neighboring edge servers or further offloaded to the cloud when edge resources become insufficient. However, we acknowledge that this does not constitute a rigorous validation of the model under explicitly controlled mobility scenarios. The extent to which the simplified abstraction affects system performance under more complex mobility conditions remains an open question, and we consider this a meaningful limitation of the current work.

VI. Conclusion

Aiming at the challenge that the lack of dynamic awareness of spatiotemporal-varying heterogeneous resources in VEC thus leads to excessively high task offloading latency under high mobility scenarios., this paper proposes a dynamic resource-aware intelligent optimization method (named DRIVE). This method constructs a cloud-edge-end collaborative computing framework to realize V2V multi-hop offloading and V2I offloading supporting service migration between edge servers. It relies on the twin critic network structure in the TD3 algorithm to achieving efficient offloading decisions with dynamic resource awareness in the multi-constrained VEC environment. Experimental results based on real datasets of base stations and taxi trajectory in Shanghai/Beijing show that DRIVE outperforms baseline methods in terms of weighted task delay, and its effectiveness and robustness meet expectations. In future work, we plan to incorporate intersection-aware traffic modeling, lane-level mobility prediction, blockage-aware communication modeling, and real-world deployment validation to further improve the practicality of the proposed framework. In addition, semantic communication and federated learning techniques will be explored to enhance cross-domain collaboration efficiency while strengthening privacy preservation.

References

[1] M. Noor-A-Rahim, Z. Liu, H. Lee, G. G. M. N. Ali, D. Pesch, and P. Xiao, "A survey on resource allocation in vehicular networks," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 701–721, Feb. 2022.

[2] S. Raza et al., "A survey on vehicular edge computing: Architecture, applications, technical issues, and future directions," *Wireless Commun. Mobile Comput.*, vol. 2019, no. 1, 2019, Art. no. 3159762.

[3] D. Zhang, F. R. Yu, R. Yang, and L. Zhu, "Software-defined vehicular networks with trust management: A deep reinforcement learning approach," *IEEE Trans. Intell. Transp. Syst.*, vol. 23, no. 2, pp. 1400–1414, Feb. 2022.

[4] J. Geng, Z. Qin, and S. Jin, "Dynamic resource allocation for cloud-edge collaboration offloading in VEC networks with diverse tasks," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 12, pp. 21235–21251, Dec. 2024.

[5] I. Sorkhoh, D. Ebrahimi, R. Atallah, and C. Assi, "Workload scheduling in vehicular networks with edge cloud capabilities," *IEEE Trans. Veh. Technol.*, vol. 68, no. 9, pp. 8472–8486, Sep. 2019, doi: 10.1109/TVT.2019.2927634.

[6] Y. Wang et al., "A game-based computation offloading method in vehicular multiaccess edge computing networks," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 4987–4996, Jun. 2020.

[7] C. Ma, J. Zhu, M. Liu, H. Zhao, N. Liu, and X. Zou, "Parking edge computing: Parked-vehicle-assisted task offloading for urban VANETs," *IEEE Internet Things J.*, vol. 8, no. 11, pp. 9344–9358, Jun. 2021.

[8] Y. Li, "Deep reinforcement learning: An overview," 2017, *arXiv:1701.07274*.

[9] S. Fujimoto et al., "Addressing function approximation error in actor-critic methods," in *Proc. Int. Conf. Mach. Learn.*, 2018, pp. 1587–1596.

[10] T. Zhou et al., "Computing offloading based on TD3 algorithm in cache-assisted vehicular NOMA–MEC networks," *Sensors*, vol. 23, no. 22, 2023, Art. no. 9064.

[11] R. Meneguette et al., "Vehicular edge computing: Architecture, resource management, security, and challenges," *ACM Comput. Surv.*, vol. 55, no. 1, pp. 1–46, 2021.

[12] Y. Dai, D. Xu, S. Maharjan, and Y. Zhang, "Joint load balancing and offloading in vehicular edge computing and networks," *IEEE Internet Things J.*, vol. 6, no. 3, pp. 4377–4387, Jun. 2019.

[13] Z. Zhang and F. Zeng, "Efficient task allocation for computation offloading in vehicular edge computing," *IEEE Internet Things J.*, vol. 10, no. 6, pp. 5595–5606, Mar. 2023.

[14] B. Cao, H. Ye, J. Liu, B. Tang, Z. Tao, and S. Deng, "SMART: Cost-aware service migration path selection based on deep reinforcement learning," *IEEE Trans. Intell. Transp. Syst.*, vol. 25, no. 9, pp. 12421–12436, Sep. 2024, doi: 10.1109/TITS.2024.3378920.

[15] Q. Qi et al., "Knowledge-driven service offloading decision for vehicular edge computing: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 68, no. 5, pp. 4192–4203, May 2019.

[16] Q. Yuan, J. Li, H. Zhou, T. Lin, G. Luo, and X. Shen, "A joint service migration and mobility optimization approach for vehicular edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 8, pp. 9041–9052, Aug. 2020.

[17] W. Zhan et al., "Deep-reinforcement-learning-based offloading scheduling for vehicular edge computing," *IEEE Internet Things J.*, vol. 7, no. 6, pp. 5449–5465, Jun. 2020.

[18] Y. Liu, Y. Li, Y. Niu, and D. Jin, "Joint optimization of path planning and resource allocation in mobile edge computing," *IEEE Trans. Mobile Comput.*, vol. 19, no. 9, pp. 2129–2144, Sep. 2020.

[19] H. Liang et al., "Reinforcement learning enabled dynamic resource allocation in the Internet of Vehicles," *IEEE Trans. Ind. Informat.*, vol. 17, no. 7, pp. 4957–4967, Jul. 2021.

[20] Y. He, N. Zhao, and H. Yin, "Integrated networking, caching, and computing for connected vehicles: A deep reinforcement learning approach," *IEEE Trans. Veh. Technol.*, vol. 67, no. 1, pp. 44–55, Jan. 2018.

[21] L. Liu, J. Feng, X. Mu, Q. Pei, D. Lan, and M. Xiao, "Asynchronous deep reinforcement learning for collaborative task computing and on-demand resource allocation in vehicular edge computing," *IEEE Trans. Intell. Transp. Syst.*, vol. 24, no. 12, pp. 15513–15526, Dec. 2023.

[22] H. Ye et al., "DELIGHT: A willingness-aware collaborative edge service offloading utilizing deep reinforcement learning," *Cluster Comput.*, vol. 28, no. 1, 2025, Art. no. 39.

[23] M. Li, J. Gao, L. Zhao, and X. Shen, "Deep reinforcement learning for collaborative edge computing in vehicular networks," *IEEE Trans. Cogn. Commun. Netw.*, vol. 6, no. 4, pp. 1122–1135, Dec. 2020.

[24] J. Shi, J. Du, J. Wang, J. Wang, and J. Yuan, "Priority-aware task offloading in vehicular fog computing based on deep reinforcement learning," *IEEE Trans. Veh. Technol.*, vol. 69, no. 12, pp. 16067–16081, Dec. 2020.

[25] T. H. Luan, X. Shen, F. Bai, and L. Sun, "Feel Bored? Join verse! Engineering vehicular proximity social networks," *IEEE Trans. Veh. Technol.*, vol. 64, no. 3, pp. 1120–1131, Mar. 2015.

[26] C. M. Huang, S. Y. Lin, and Z. Y. Wu, "The k-hop-limited V2V2I VANET data offloading using the mobile edge computing (MEC) mechanism," *Veh. Commun.*, vol. 26, 2020, Art. no. 100268.

[27] A. B. de Souza et al., "A context-oriented framework for computation offloading in vehicular edge computing using WAVE and 5G networks," *Veh. Commun.*, vol. 32, 2021, Art. no. 100389.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.

[28] W. Zhan, C. Luo, G. Min, C. Wang, Q. Zhu, and H. Duan, "Mobility-aware multi-user offloading optimization for mobile edge computing," *IEEE Trans. Veh. Technol.*, vol. 69, no. 3, pp. 3341–3356, Mar. 2020.
[29] J. Feng, F. R. Yu, Q. Pei, X. Chu, J. Du, and L. Zhu, "Cooperative computation offloading and resource allocation for blockchain-enabled mobile-edge computing: A deep reinforcement learning approach," *IEEE Internet Things J.*, vol. 7, no. 7, pp. 6214–6228, Jul. 2020.
[30] J. H. Holland, "Genetic algorithms," *Sci. Amer.*, vol. 267, no. 1, pp. 66–73, 1992.
[31] I. Aydin, M. Karakose, and E. Akin, "A multi-objective artificial immune algorithm for parameter optimization in support vector machine," *Appl. Soft Comput.*, vol. 11, no. 1, pp. 120–129, 2011.
[32] V. Mnih et al., "Human-level control through deep reinforcement learning," *Nature*, vol. 518, no. 7540, pp. 529–533, 2015.
[33] T. P. Lillicrap et al., *Continuous Control with Deep Reinforcement Learn.*, pp. 1–10, 2015.
[34] A. Akbar et al., "SAVE: Self-aware vehicular edge computing with efficient resource allocation," in *Proc. IEEE Int. Conf. Autonomic Comput. Self-Organizing Syst.*, 2023, pp. 157–162.
[35] X. Dai et al., "A learning-based approach for vehicle-to-vehicle computation offloading," *IEEE Internet Things J.*, vol. 10, no. 8, pp. 7244–7258, Apr. 2023.
[36] C. Wang et al., "Joint computation offloading and resource allocation for end-edge collaboration in Internet of Vehicles via multi-agent reinforcement learning," *Neural Netw.*, vol. 179, 2024, Art. no. 106621.
[37] F. G. Wakgra, W. Yahya, B. Kar, Y.-C. Lai, Y.-D. Lin, and S. B. Tadele, "Ratio-based offloading optimization for edge and vehicular-fog federated systems: A multi-agent TD3 approach," *IEEE Trans. Veh. Technol.*, vol. 73, no. 11, pp. 17684–17696, Nov. 2024.
[38] J. Yan, S. Bi, and Y. J. A. Zhang, "Offloading and resource allocation with general task graph in mobile edge computing: A deep reinforcement learning approach," *IEEE Trans. Wireless Commun.*, vol. 19, no. 8, pp. 5404–5419, Aug. 2020.



Buqing Cao received the PhD degree in computer science from Wuhan University, in 2011 and the Post-doctoral degree from the Department of Computer Science and Computer Engineering, University of Arkansas, Fayetteville, AR, USA, in 2016. He is currently a professor with the Hunan University of Science and Technology, Xiangtan, China and the School of Computer Science and Artificial Intelligence, Hunan University of Technology, Zhuzhou, China. His current research interests include recommendation system, intelligent transportation system and machine learning. He has authored or coauthored more than 100 papers in well-known conferences and journals, such as *IEEE Transactions on Services Computing*, *IEEE Transactions on Network and Service Management*, IST and Icws. He is on the editorial boards of several journals and has served as a program committee member for multiple international conferences.



Zhi Tao is currently working toward the master's degree with the Hunan Key Laboratory for Service Computing and Novel Software Technology, School of Computer Science and Engineering, Hunan University of Science and Technology, Xiangtan, China. His research interests include mobile edge computing and recommendation systems.



Shanpeng Liu received the PhD degree in software engineering from the Hunan University of Science and Technology, Xiangtan, China, in 2025. He is currently a researcher with the School of Computer Science and Artificial Intelligence, Hunan University of Technology, Zhuzhou, China. He has authored or coauthored more than 18 papers in international journals and conferences, including TIFS, TITS, TNSM, TBD, TAI, JBHI, IoTJ, KBS, and ICWS. His research interests include service recommendation, mobile computing, and privacy computing.



Hongfan Ye is currently a faculty member with the School of Computer and Artificial Intelligence, Huaihua University. His research focuses on mobile edge computing. He has authored or coauthored more than 10 papers in international journals and conferences such as *IEEE Transactions on Intelligent Transportation Systems*, *IEEE Transactions on Network and Service Management*, *Journal of Cloud Computing, Cluster Computing*, IJCSE, ISPA2019, ICA3PP2022, CSCWD2023 and so on.



Jinjun Chen (Fellow, IEEE) received the Ph.D. degree in information technology from the Swinburne University of Technology, Australia. He is currently a professor with the Swinburne University of Technology. He is the deputy director of Swinburne Data Science Research Institute. He has authored or coauthored in more than 300 papers in international journals and conferences, including various IEEE/ACM Transactions in his research interests which include scalability, Big Data, data science, data systems, cloud computing, data privacy and security, health data analytics, and related various research topics.

Authorized licensed use limited to: Indian Institute of Petroleum and Energy. Downloaded on October 26,2026 at 11:58:57 UTC from IEEE Xplore. Restrictions apply.